

Digital Image Processing

Homework 1

{Image input/flip/output + Resolution + Cropping}

The BMP (Bitmap) format is a widely used file format for storing digital images. It is a straightforward format that stores pixel data in a grid, where each pixel's color is represented either directly (in true color images) or through a palette (in indexed color images). The BMP format is also known for being uncompressed, which results in larger file sizes but ensures that image quality is preserved.

A BMP file consists of four main sections that are stored sequentially within the file:

1. File Header (14 bytes), which contains general information, such as its size and the location where the pixel data begins.
2. DIB (Device Independent Bitmap) header, typically 40 bytes in size, provides detailed information about the image, including its width, height, color depth, and compression method.
3. Color Palette (based on Bit Depth), which contains a list of colors referenced by the pixel data. For each pixel, a value from the color table is used to determine its actual color. The size of the color table depends on the bit depth.
4. Pixel Data (the actual image) depends on the image's `biHeight` value to determine the order. If `biHeight` is positive, the pixel data starts from the bottom-left corner of the image and moves upward. Conversely, if `biHeight` is negative, the pixel data begins at the top-left corner and moves downward. Each row of pixel data is padded to a multiple of 4 bytes to align memory, ensuring efficient reading and writing.

Summary:

The BMP format is simple, widely supported, and suitable for tasks that prioritize image fidelity, as it provides high-quality, uncompressed images. However, the lack of compression leads to large file sizes, making it less ideal when storage space is a concern. It is commonly used in applications like medical imaging or graphic design, where uncompressed or lossless image storage is preferred, though it offers limited support for advanced features like transparency.

I. Image input/flip/output

(1) Define Two Structures to Hold BMP Headers:

`FileHeader` and `InfoHeader`, which store metadata about the BMP file such as file size, image dimensions, and pixel format (e.g., 24-bit or 32-bit depth). These structures are used to read and write header information from and to BMP files.

(2) Flip Horizontally:

The `pixel\_swap` function performs the essential operation of flipping an image horizontally. It accepts a vector called `pixelData`, which contains the raw pixel data of the image, along with parameters for image width, height, bytes per pixel, and the row size (which includes any necessary padding). The function iterates through each row of the image, swapping the left and right pixels based on their indices. For 24-bit and 32-bit BMP images, pixels are stored in groups of 3 or 4 bytes, respectively, representing the RGB (and optionally Alpha) color channels. By swapping the pixels within each row, the image is effectively flipped horizontally.

(3) Modify Pixel Data:

In the `flip\_horizontal` function, the input BMP file is opened in binary mode, and the BMP headers are read to extract details about the image, such as its width, height, and bit depth. The pixel data is read into a vector, and if the bit depth is supported (either 24-bit or 32-bit), the `pixel\_swap` function is called to modify the pixel data. Finally, the headers and the modified pixel data are written to an output BMP file.

Summary:

The code reads and processes BMP files by flipping them horizontally, swapping the pixel values in each row while preserving the image's color information. It operates efficiently with minimal memory usage by utilizing file streams, which directly load and modify the binary pixel data. Furthermore, it supports common bit depths of 24-bit and 32-bit, ensuring compatibility with a wide range of BMP formats.

II. Resolution

(1) Quantization Process:

The ``quantize_value`` function reduces the color depth of each pixel. This function takes an 8-bit color channel value and scales it down to a specified target bit depth (e.g., 6-bit, 4-bit, or 2-bit). The quantization levels are determined based on the target bit depth, and the pixel value is adjusted to align with these levels. This process diminishes the overall color detail of the image, effectively compressing it visually while maintaining the BMP format.

(2) Pixel Manipulation:

For each pixel in the image, the ``quantization`` function applies the quantization process to the red, green, and blue channels, as well as the alpha channel if it is present in 32-bit images. The pixel values are accessed through their positions in the ``pixelData`` vector, and only the color channels are modified, while the alpha channel (transparency) remains unchanged.

(3) Handling Different Bit Depths:

It supports both 24-bit (RGB) and 32-bit (RGBA) BMP images, determining the number of bytes per pixel based on the bit depth. This functionality enables the code to manage images with or without an alpha channel and to apply the appropriate color quantization accordingly.

Summary:

This code efficiently handles BMP image quantization by directly manipulating pixel data, headers, and file streams. It supports multiple bit depths and processes each pixel while preserving the structure of the BMP file format. The quantization process reduces the color depth of each pixel channel (red, green, and blue), enabling image compression at various bit levels (e.g., 6-bit, 4-bit, or 2-bit) without affecting the alpha channel in 32-bit images. Additionally, the code can process multiple BMP files in batches, applying different levels of quantization to each file and automatically generating output files.

III. Cropping

(1) Adjusting the Crop Region:

After reviewing the headers, the ``cropping`` function verifies the specified crop dimensions (defined by `cropX`, `cropY`, `cropWidth`, and `cropHeight`) to ensure they do not exceed the original image size. If the dimensions exceed the original size, they are adjusted to fit within the image's boundaries.

(2) Row Size Calculation:

BMP images store pixel data in a row-by-row format, with each row padded to align with a 4-byte boundary according to BMP format specifications. The code calculates the padded size of each row for both the original and cropped images, ensuring that both the reading and writing processes take this padding into account.

(3) Updating Headers for the Cropped Image:

The ``InfoHeader`` and ``FileHeader`` are updated to accurately reflect the dimensions and size of the cropped image. The width and height are adjusted to match the crop dimensions, and the file size is recalculated based on the new pixel data size.

(4) Reading, Cropping, and Writing Image Data:

The loop in the ``cropping`` function reads pixel data from the original image row by row. If the current row falls within the vertical bounds of the crop (from ``cropY`` to ``cropY` + `cropHeight``), it extracts the horizontal range of pixels specified by ``cropX`` and ``cropWidth``. Depending on the image's bit depth—either 24-bit or 32-bit—the function extracts the appropriate number of bytes per pixel (3 bytes for 24-bit and 4 bytes for 32-bit). It then writes the cropped row to the output BMP file.

(5) Set the Crop Parameters:

The cropping parameters for each input image can be customized in the ``main`` function, including ``cropX``, ``cropY``, ``cropWidth``, and ``cropHeight``. After setting these parameters, call the ``cropping`` function to execute the cropping operation.

Summary:

The code efficiently crops a BMP image by adjusting its dimensions and padding, while ensuring that both the headers and pixel data of the cropped image are handled correctly.